

Our real product is a client app running on IoT devices, a map with live layers (cameras, video, sensor/lidar data), where devices talk to each other in a mesh with no fixed host. This task is a scaled-down version that touches the same core problems: real-time data over sockets, rendering live-updating visual state, and handling data that's incomplete, delayed, duplicated or conflicting. You don't need to build a mesh network.

Time budget: 1-2 evenings. You won't have time for everything. Prioritise in this order: (1) core + ordering, (2) throttling, (3) reconnect/resume, (4) optional stretch. If you run out of time, cut scope and say so in the README. We'd rather see fewer things done well than everything half-working.

1. Backend (Node.js + WebSocket)

Simulate 5-8 virtual devices, hardcoded in-process (no database). Each device, on its own interval:

- Emits a position update (x/y in an arbitrary 2D space) and a fake sensor reading.
- Occasionally goes offline for a stretch and comes back. While offline it sends nothing.
- Occasionally **reboots**: its sequence counter restarts from 0 and it gets a new `bootId`.
- Occasionally emits a discrete **event** (e.g. `low_battery`, `alert`) in addition to the regular state updates.

Every message carries `deviceId`, `bootId`, a per-device `seq`, a device-side timestamp `ts` (give each device a slightly different clock offset), and a payload. Message types are `state` and `event`.

Network chaos layer. Between the devices and the socket, add a layer that randomly delays some messages (100 ms to 3 s) and duplicates a few, so delivery order differs from emission order. Make it configurable (env vars or a small control endpoint).

Rate and stress mode. Normal operation is about 5-10 messages/sec per device. Add a stress mode (toggled via an HTTP endpoint or a WS control message) that pushes the aggregate to 500+ messages/sec.

Protocol.

- On connect, the client receives a snapshot (latest known state per device), then live messages.
- The client can send a `subscribe` message specifying which devices it wants and a `maxHz` limit per device. The server must honour it by **throttling**.
- A slow client must not make server memory grow without bound.
- Keep a short per-device history on the server. A reconnecting client sends its last seen `seq` per device, and the server either replays the gap or sends a fresh snapshot if the gap is too large.
- Provide a way to force-drop all client connections (e.g. `POST /debug/drop-connections`) so reconnects can be tested.

2. Frontend (React)

- Connect over WebSocket and render devices on a 2D canvas (`<canvas>`, SVG or Three.js) as moving points.
- Visually distinguish **online** / **stale** / **offline**.
- Give one device a richer live sensor visualization (real-time chart, pulsing indicator, etc.). This is where we want to see rendering care.

- **Connection layer:** keep the socket outside React component state. Reconnect with exponential backoff and jitter. Show connection status in the UI. A silent device and a lost connection must look different to the user.
- **Render performance:** rendering should follow the display rate, not the message rate. The UI must stay smooth in stress mode.
- **Diagnostics overlay:** a small on-screen readout of messages/sec received, frames/sec rendered, and how many messages were dropped as duplicate, out-of-order or coalesced.
- **Throttle control:** a UI control for maxHz that sends the `subscribe` message.

3. Decisions we're deliberately not specifying

Decide, implement and document your reasoning for each:

- **Reconnect gap.** When a device comes back after being offline, do you show its last known position as-is, interpolate, flag it as stale, or something else?
- **Ordering.** Delayed and duplicated messages will arrive. How does the frontend avoid showing stale data as current? What do you trust for ordering (`seq`, `device ts`, server receive time), given clock offsets and device reboots?
- **What to drop under load.** Which data can be coalesced (latest wins) and which must never be dropped? Where does throttling happen (server, client, both)?
- **Resume vs snapshot.** When do you replay and when do you resync? Where's your cutoff?
- **Real mesh.** In your README, a short paragraph: what in your design would break or need to change when there is no central server to hold the history buffer?

4. Optional stretch (not expected)

Run two gateway processes on different ports, each seeing an overlapping subset of devices with different latencies. The client connects to one, fails over if it dies, and merges state from both. Describing how you'd do this in the README is fine.

Tests

Write a few unit tests for your ordering/reconciliation logic (duplicates, out-of-order, gap, reboot). No end-to-end tests required.

Submission

Public/private repo (invite us if private) or a zip, with `npm install && npm run dev`-style setup (a monorepo is fine). The README must include:

- run instructions and how to trigger stress mode and connection drops
- your decisions from section 3 and why
- what you cut and why
- what you'd do with more time